

Week 6 — Day 4

Transaction Rollback Test Documentation

This document records the complete Transaction Rollback test scenario in the actual chronological order of execution.

1. Test Objective

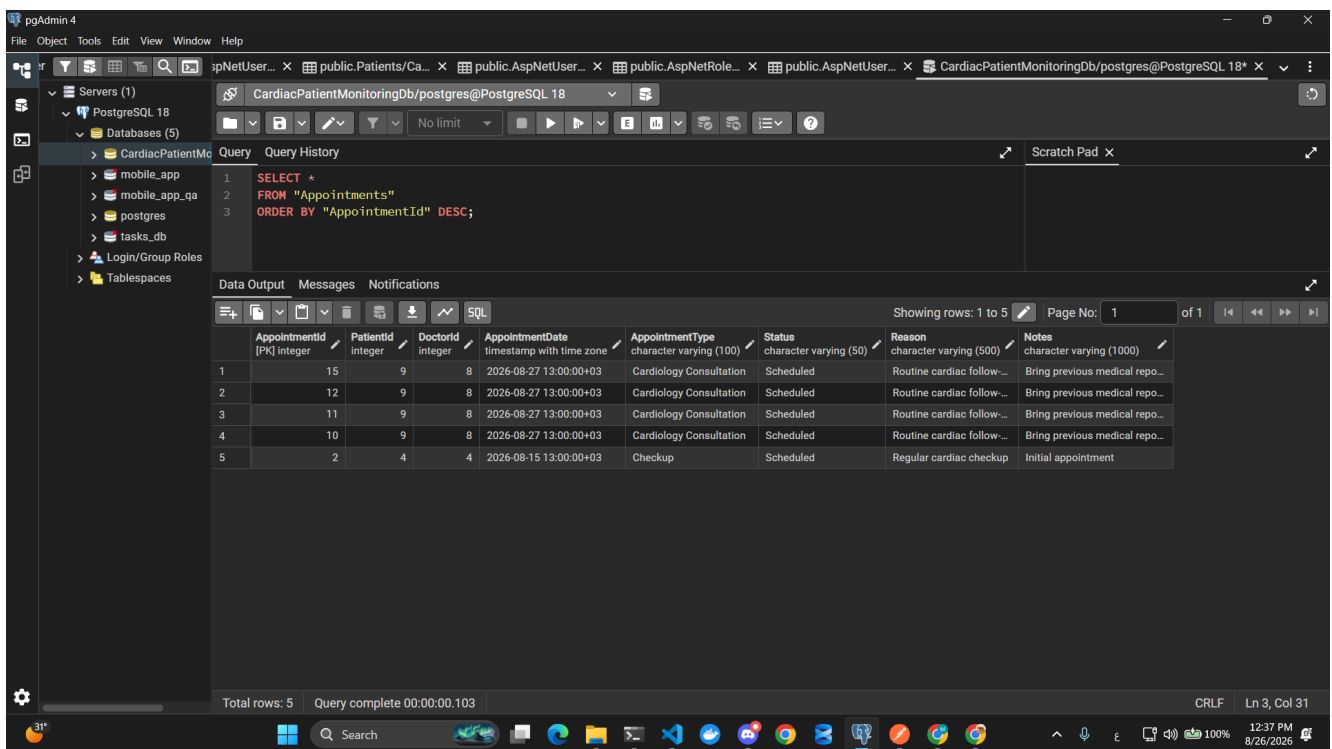
The objective was to verify that creating an Appointment and its related Notifications is handled inside a single database transaction. If an exception occurs before the transaction is committed, all changes made during the operation must be rolled back.

2. Test Flow

Before Test → Create Appointment → Add Notifications → Throw Exception → HTTP 500 → Rollback → Verify Data Was Removed → Remove Test Exception → Retry → HTTP 201 Created → Verify Data Was Saved.

3. Evidence — Chronological Order

Figure 1 — Before the rollback test: existing Appointment records are checked in PostgreSQL.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'CardiacPatientMonitoringDb'. The main query editor shows a SQL query to select all records from the 'Appointments' table, ordered by 'AppointmentId' in descending order. The results pane shows 5 rows of data.

AppointmentId [PK] integer	PatientId integer	DoctorId integer	AppointmentDate timestamp with time zone	AppointmentType character varying (100)	Status character varying (50)	Reason character varying (500)	Notes character varying (1000)
1	15	9	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
2	12	9	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
3	11	9	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
4	10	9	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
5	2	4	2026-08-15 13:00:00+03	Checkup	Scheduled	Regular cardiac checkup	Initial appointment

Figure 2 — Before the rollback test: existing Notification records are checked in PostgreSQL.

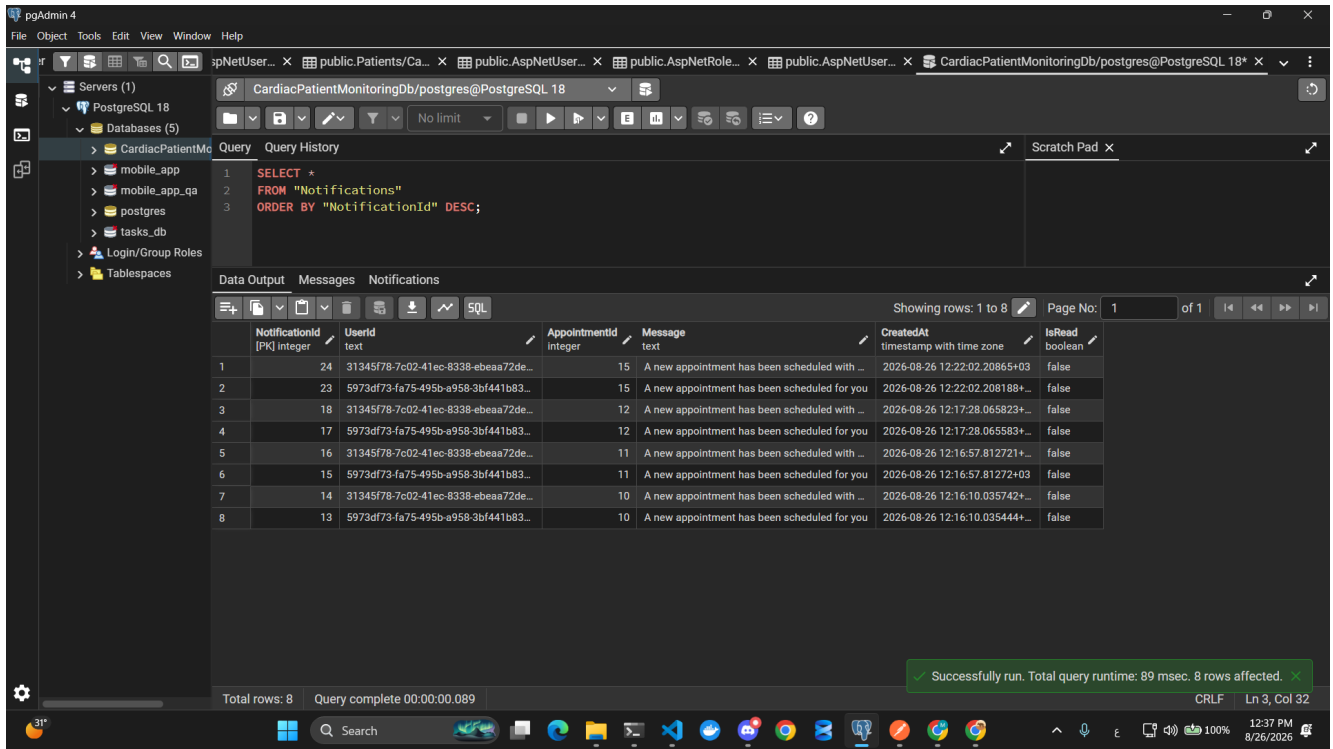


Figure 3 — The Create Appointment request is sent and returns HTTP 500 Internal Server Error.

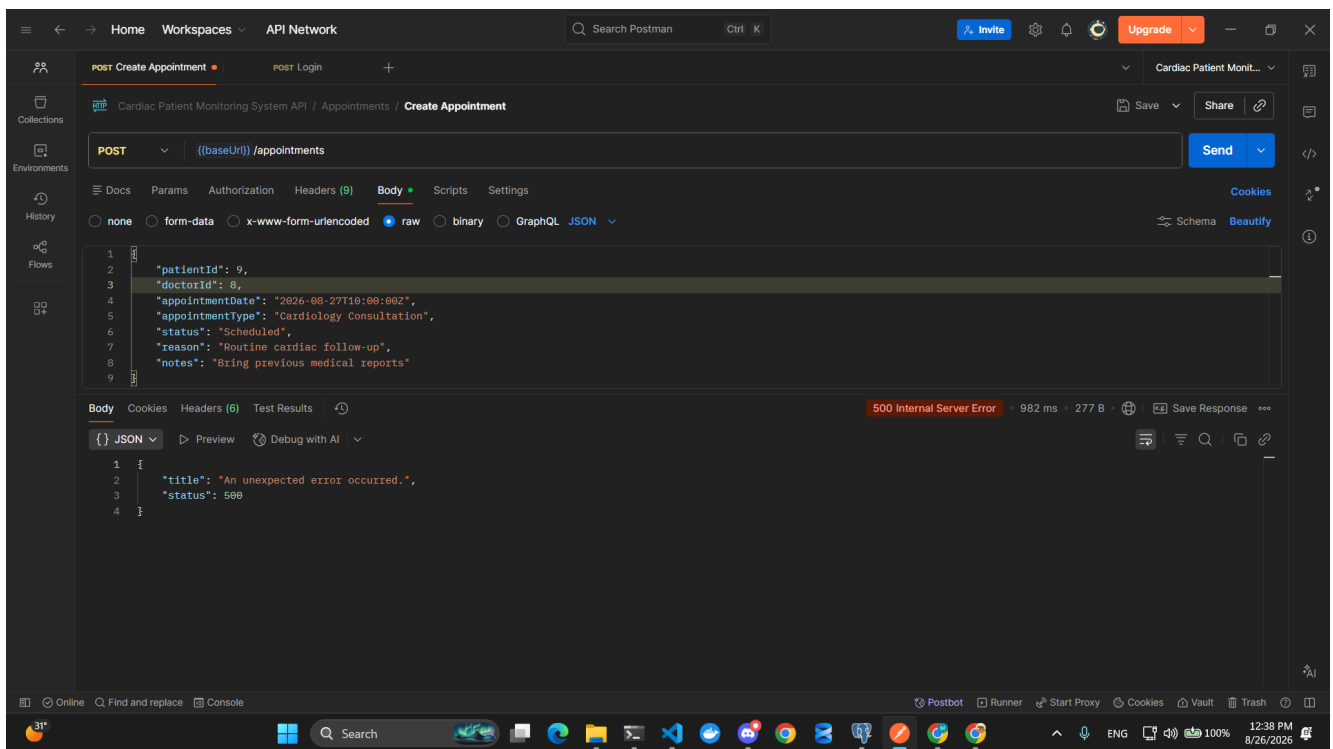


Figure 4 — The application terminal shows the intentional exception: Testing transaction rollback.

```
PS C:\Backend Internship\HelloBinX\Tasks\Week 6\Day 4\Cardiac-Patient-Monitoring-System\CardiacPatientMonitoringSystem> dotnet run
VALUES (epo, epo, ep/, epo, epo/)
RETURNING "NotificationId";
fail: CardiacPatientMonitoringSystem.Middleware.ExceptionHandlingMiddleware[0]
Unhandled exception occurred while processing /api/appointments
System.Exception: Testing transaction rollback
at CardiacPatientMonitoringSystem.Services.AppointmentService.CreateAsync(CreateAppointmentDto dto, String userId, Boolean isAdmin, Boolean isDoctor) in C:\Backend Internsh
at CardiacPatientMonitoringSystem.Services.AppointmentService.CreateAsync(CreateAppointmentDto dto, String userId, Boolean isAdmin, Boolean isDoctor) in C:\Backend Internsh
ip\HelloBinX\Tasks\Week 6\Day 4\Cardiac-Patient-Monitoring-System\CardiacPatientMonitoringSystem\Services\AppointmentService.cs:line 183
at CardiacPatientMonitoringSystem.Services.AppointmentService.CreateAsync(CreateAppointmentDto dto, String userId, Boolean isAdmin, Boolean isDoctor) in C:\Backend Internsh
ip\HelloBinX\Tasks\Week 6\Day 4\Cardiac-Patient-Monitoring-System\CardiacPatientMonitoringSystem\Services\AppointmentService.cs:line 184
at CardiacPatientMonitoringSystem.Controllers.AppointmentsController.Create(CreateAppointmentDto dto) in C:\Backend Internship\HelloBinX\Tasks\Week 6\Day 4\Cardiac-Patien
Monitoring-System\CardiacPatientMonitoringSystem\Controllers\AppointmentsController.cs:line 69
at lambda_method178(Closure, Object)
at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.AwaitableObjectResultExecutor.Execute(ActionContext actionContext, IActionResultTypeMapper mapper, ObjectMet
hodExecutor executor, Object controller, Object[] arguments)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.<InvokeActionMethodAsync>g__Awaited|12_0(ControllerActionInvoker invoker, ValueTask`1 actionResultValueTa
sk)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.<InvokeNextActionFilterAsync>g__Awaited|10_0(ControllerActionInvoker invoker, Task lastTask, State next,
Scope scope, Object state, Boolean isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Rethrow(ActionExecutedContextSealed context)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State& next, Scope& scope, Object& state, Boolean& isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.<InvokeInnerFilterAsync>g__Awaited|13_0(ControllerActionInvoker invoker, Task lastTask, State next, Scope
scope, Object state, Boolean isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeFilterPipelineAsync>g__Awaited|20_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope, Objec
t state, Boolean isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
at Microsoft.AspNetCore.Authorization.AuthorizationMiddleware.Invoke(HttpContext context)
at Microsoft.AspNetCore.Authentication.AuthenticationMiddleware.Invoke(HttpContext context)
at Program.<>c.<<Main>$>b__0_6.MoveNext() in C:\Backend Internship\HelloBinX\Tasks\Week 6\Day 4\Cardiac-Patient-Monitoring-System\CardiacPatientMonitoringSystem\Program.
cs:line 164
--- End of stack trace from previous location ---
at CardiacPatientMonitoringSystem.Middleware.ExceptionHandlingMiddleware.InvokeAsync(HttpContext context) in C:\Backend Internship\HelloBinX\Tasks\Week 6\Day 4\Cardiac-Pati
ent-Monitoring-System\CardiacPatientMonitoringSystem\Middleware\ExceptionHandlingMiddleware.cs:line 24
```

Figure 5 — After the failed transaction: the Appointment is checked and the failed Appointment is not persisted.

The screenshot displays the pgAdmin 4 interface. The left sidebar shows the server hierarchy: Servers (1) > PostgreSQL 18 > Databases (5) > CardiacPatientMonitoringDb. The main pane shows the 'Query' tab with the following SQL query:

```
1 SELECT *
2 FROM "Appointments"
3 ORDER BY "AppointmentId" DESC;
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query in a table. The table has 9 columns: AppointmentId (PK integer), PatientId integer, DoctorId integer, AppointmentDate timestamp with time zone, AppointmentType character varying (100), Status character varying (50), Reason character varying (500), and Notes character varying (1000). The table shows 5 rows of data.

AppointmentId (PK integer)	PatientId integer	DoctorId integer	AppointmentDate timestamp with time zone	AppointmentType character varying (100)	Status character varying (50)	Reason character varying (500)	Notes character varying (1000)	
1	15	9	8	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
2	12	9	8	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
3	11	9	8	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
4	10	9	8	2026-08-27 13:00:00+03	Cardiology Consultation	Scheduled	Routine cardiac follow...	Bring previous medical repo...
5	2	4	4	2026-08-15 13:00:00+03	Checkup	Scheduled	Regular cardiac checkup	Initial appointment

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.093'. The bottom of the image shows the Windows taskbar with various application icons and the system clock showing 12:39 PM on 8/26/2026.

Figure 6 — After the failed transaction: the related Notifications are checked and are not persisted.

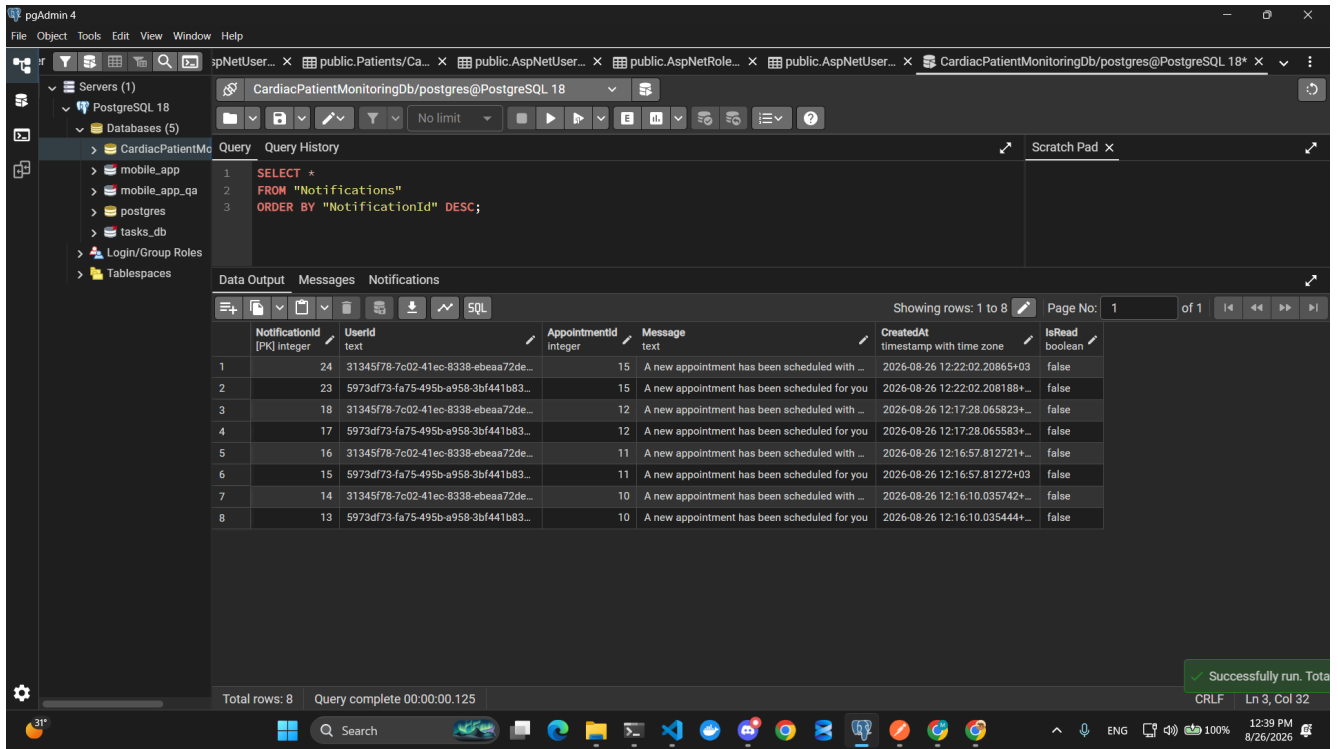


Figure 7 — After removing the test exception: the same request succeeds with HTTP 201 Created and AppointmentId 17.

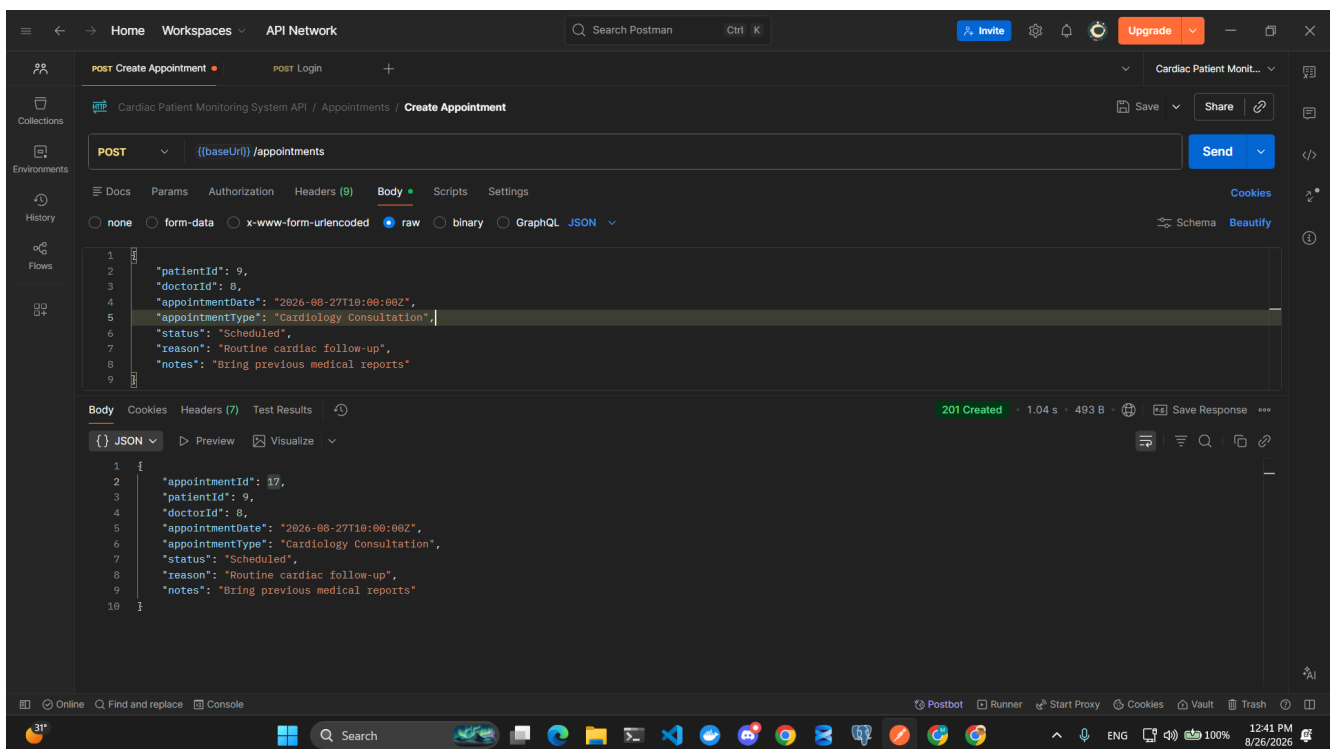


Figure 8 — After the successful retry: AppointmentId 17 is present in the Appointments table.

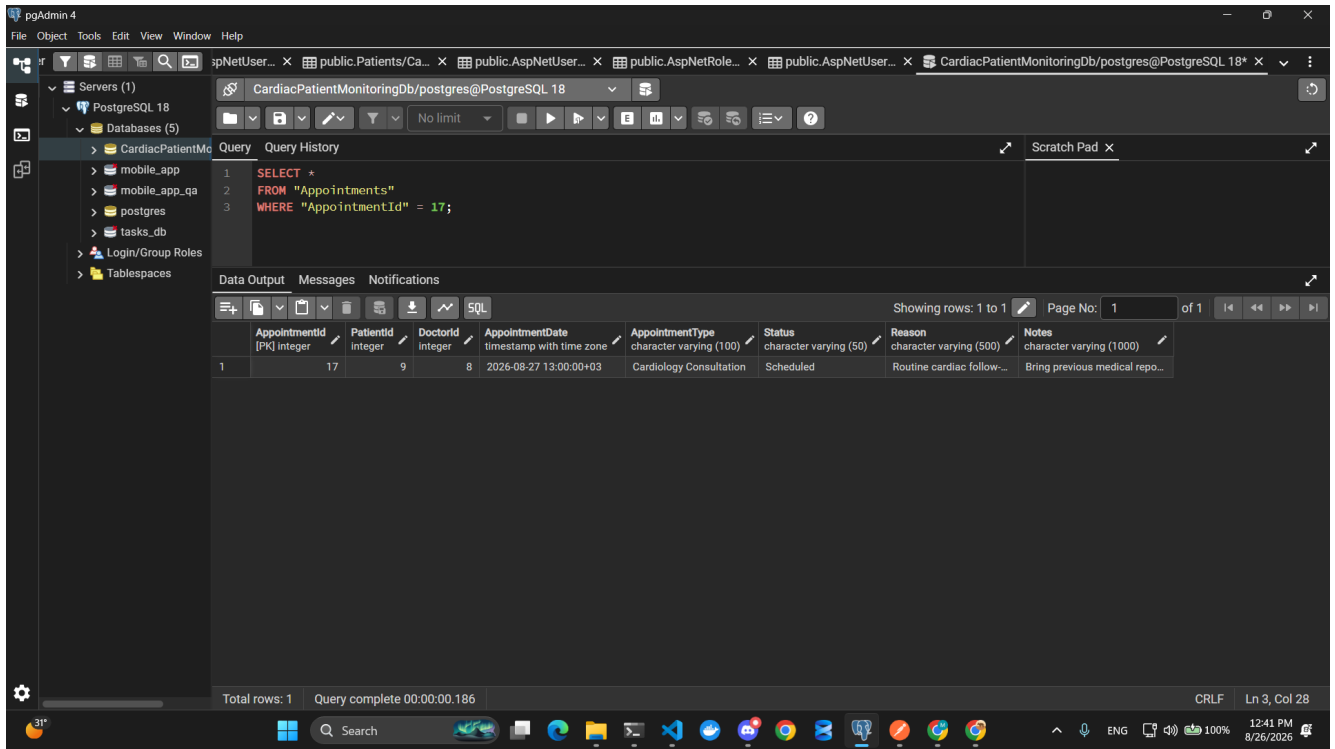
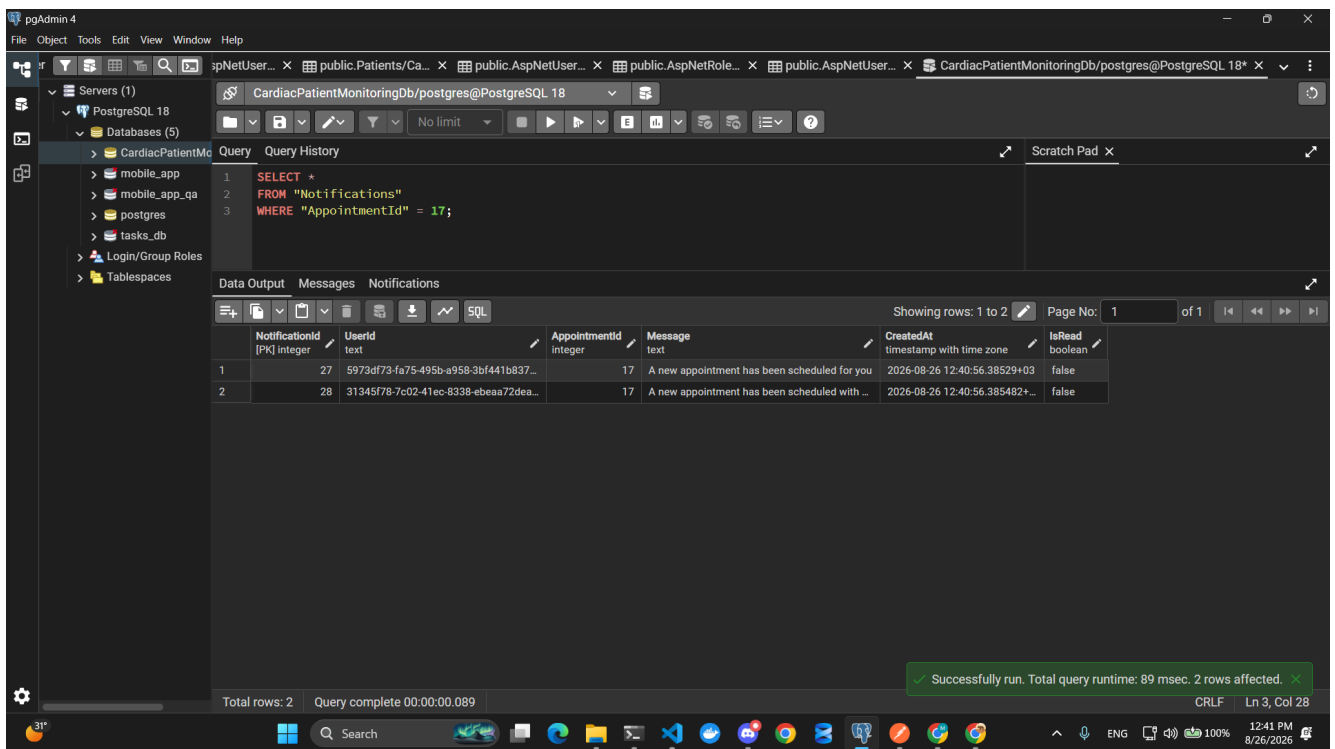


Figure 9 — After the successful retry: Notifications for AppointmentId 17 are present.



4. Intentional Failure

The rollback test was performed by intentionally throwing an exception after the Appointment and Notifications had been added but before the transaction was committed:

```
throw new Exception("Testing transaction rollback");
```

Postman returned HTTP 500 Internal Server Error, while the terminal displayed the exception and stack trace.

5. Rollback Verification

The database was checked after the failed request. The Appointment created during the failed transaction was not persisted, and the Notifications associated with that transaction were also not persisted.

This demonstrates that the transaction rolled back the changes made before the exception.

6. Successful Retry

The intentional throw statement was then removed. The application was restarted and the same request was executed again.

The request returned HTTP 201 Created. AppointmentId 17 was created successfully, and the related Notifications were stored in the database.

7. Final Result

The test confirmed atomic behavior: either the Appointment and its Notifications are committed together, or an exception causes the entire operation to roll back.

8. Conclusion

The Transaction implementation successfully prevents partial database changes. This keeps the Appointment and Notification data consistent even when an error occurs during the operation.